Étude de cas 1 : Lecture automatique des chèques

Jalon 3 : Un nouveau scanner, deux tâches

chapitre 3 - transfert et multitâche

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

Objectif du jalon

Adapter le lecteur au nouveau scanner avec seulement 600 images étiquetées, et faire cohabiter lecture du montant et détection d'altération dans un seul modèle.

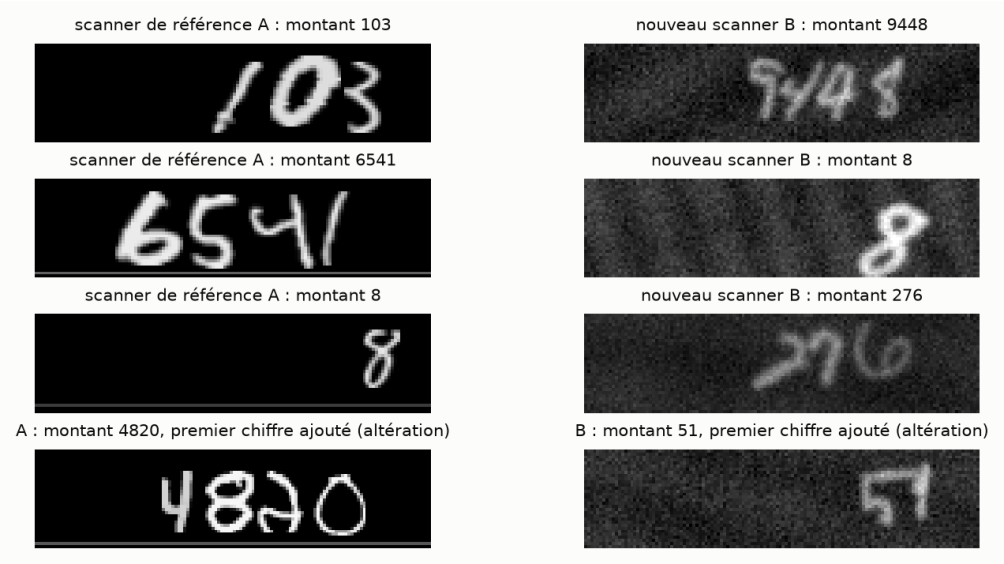
Jalon autonome : ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc1_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
Environnement : Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

1 Contexte

La banque fictive *Banque Atlas* traite chaque jour des dizaines de milliers de chèques. Le montant en chiffres, écrit à la main dans un champ cadré à droite, doit être lu automatiquement ; lorsque la machine n'est pas assez sûre d'elle, le chèque part vers un opérateur. La banque veut automatiser le plus possible **sans dépasser 1 % d'erreur** sur les chèques traités automatiquement.

Certains chèques sont **altérés** : un fraudeur ajoute un chiffre devant le montant ou retouche un chiffre existant, avec un tracé légèrement différent. Enfin, les agences reçoivent un **nouveau scanner** dont les images sont plus pâles, floues et portent un motif de sécurité ; seules 600 images de ce scanner sont étiquetées.

L'étude de cas en cinq jalons : un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).



2 Données et cibles de ce jalon

Fichier	Contenu
`uc1_domaineA.npz`	scanner de référence : 15 000 / 3 000 / 5 000 images (entraînement, validation, test)
`uc1_domaineB.npz`	nouveau scanner : 600 / 400 / 3 000 images

Chaque image, en niveaux de gris, mesure 32×128 pixels (encre claire sur fond sombre). Elle est accompagnée du montant écrit (chaîne de 1 à 5 chiffres), d'un indicateur d'altération et du type d'altération. Les chiffres proviennent du jeu public MNIST ; les montants, leur mise en page, les altérations et les défauts du scanner sont simulés. Les ensembles sont imposés et n'ont aucune image de chiffre en commun.

Cibles du cahier des charges concernées

Exigence	Mesure (ensemble de test)	Cible
Lire le montant (scanner B)	part des montants entièrement exacts	$\geq 0{,}70$
Automatiser (scanner B)	même mesure	$\geq 0{,}15$
Repérer les altérations (scanner A)	AUC	$\geq 0{,}95$

Le **taux d'automatisation** se calcule en triant les chèques par confiance décroissante : c'est la plus grande part traitée automatiquement telle qu'au moins 99 % de ces lectures soient exactes (fonction `taux_automatisation` du notebook). Ces cibles décrivent le système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

3 Point de départ

Le notebook de démarrage fournit les données des deux scanners, l'encodage par positions et les métriques. Le code ci-dessous entraîne le **lecteur de départ** de ce jalon : un CNN à cinq têtes appris sur le scanner A (trois blocs convolution-pooling de 32, 64 et 64 filtres, puis une couche

dense de 128 neurones). Si vous disposez déjà d'un meilleur lecteur, vous pouvez le remplacer, à condition de le décrire.

Code de démarrage (fichier `uc1_jalon3_depart.py`)

```
# Étude de cas 1 - Jalon 3 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.
# Lecteur de départ : CNN à cinq têtes entraîné sur le scanner A.

def cnn_lecteur():
    entree = keras.Input((32, 128, 1))
    x = entree
    for filtres in (32, 64, 64):
        x = layers.Conv2D(filtres, 3, padding="same", activation="relu")(x)
        x = layers.MaxPooling2D(2)(x)
    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu", name="dense_lecture")(x)
    x = layers.Dropout(0.3)(x)
    sorties = {f"pos{k}": layers.Dense(11, activation="softmax",
                                       name=f"pos{k}")(x)
               for k in range(L_MAX)}
    return keras.Model(entree, sorties, name="cnn_lecteur")

PERTES = {f"pos{k}": "sparse_categorical_crossentropy" for k in range(L_MAX)}
lecteur_A = cnn_lecteur()
lecteur_A.compile(keras.optimizers.Adam(1e-3), loss=PERTES)
lecteur_A.fit(XA["train"], en_dict(YA["train"]), epochs=15, batch_size=128,
              validation_data=(XA["val"], en_dict(YA["val"])), verbose=2)

# Cibles du scanner B : 600 images étiquetées pour l'entraînement
YB = {s: vers_positions(B["montant_" + s]) for s in ["train", "val", "test"]}
for dom, X, D in [("A", XA, A), ("B", XB, B)]:
    lus, conf = lire_positions(lecteur_A, X["val"])
    print(dom, evaluer_lecture(D["montant_val"], lus, conf))
```

4 Travail demandé

Jalon 3 : Un nouveau scanner, deux tâches

Notions mobilisées : décalage de covariables, entraînement de zéro, extraction de caractéristiques, ajustement fin, augmentation de données, apprentissage multitâche, pondération des pertes.

1. Mesurer la chute du lecteur de départ sur le scanner B, puis caractériser le décalage entre les deux scanners (intensités, flou, motif de fond, rotation).
2. Comparer trois stratégies sur les 600 images étiquetées de B : entraînement de zéro, extraction de caractéristiques (couches convolutives gelées), ajustement fin. Faire varier le taux d'apprentissage de l'ajustement fin.
3. Tester une augmentation de données qui imite le scanner B pendant l'entraînement sur A, seule puis combinée à l'ajustement fin. Quelle stratégie est la plus efficace, et à quel coût ?

4. Construire un modèle multitâche : lecture du montant (cinq têtes) et détection d'altération (une sortie) sur un tronc commun. Étudier la pondération λ de la tâche d'altération. La détection se transfère-t-elle au scanner B ?

Vérification théorique (à la main, puis vérifiée par le code)

Pour le lecteur de départ ci-dessous, comptez le nombre total de paramètres, puis ceux qui restent entraînables si l'on gèle les couches convolutives. Quelle proportion du modèle cela représente-t-il ? Qu'en déduisez-vous pour un apprentissage sur 600 images, et quelle modification d'architecture proposeriez-vous pour rendre l'extraction de caractéristiques plus utile ?

À justifier dans votre page de décisions

La procédure à suivre à la prochaine livraison de scanners ; le λ retenu et son effet sur les deux tâches.

5 À rendre à la fin du chapitre

Livrables du jalon 3

- le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
- le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=3`), y compris les essais infructueux ;
- une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

Protocole : tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.